

Project Specifications

Doctor Clinic DevOps System

Specifications Document

Muhammed Munir Al Tawil · DevOps Course · May 2026

1. Project Context

Doctor Clinic DevOps System is a Laravel-based clinic management and appointment booking platform. The DevOps delivery layer transforms the original application into an operationally ready, containerized, and automatically tested system.

The project covers the full DevOps lifecycle: containerization, CI/CD automation, live deployment on Proxmox Virtual Environment, backup procedures, monitoring planning, and Infrastructure as Code foundations.

2. Business Need

Healthcare systems require reliability, repeatability, and fast recovery. Manual deployment increases operational risk. This project addresses that by:

- Guaranteeing deployment consistency across environments
- Automating validation before every delivery
- Providing operational readiness through backups and monitoring
- Delivering live access via a public URL on a cloud-hosted Proxmox server

3. Objectives

- Containerize the Laravel clinic application with Docker
- Create a reproducible environment with Docker Compose (app + nginx + mysql)
- Automate build, test, and delivery with GitLab CI/CD (5 stages)
- Deploy live on Proxmox VE 8.4 with public internet access
- Prepare Kubernetes manifests for future cluster deployment
- Provide database backup automation and recovery documentation
- Document monitoring strategy and operational controls
- Deliver Terraform Infrastructure as Code starter files

4. Functional Scope

In Scope	Out of Scope
Laravel application runtime, Docker, Compose, CI/CD pipelines, Kubernetes manifests, backups, monitoring plan, Terraform starter, Proxmox live deployment, delivery documentation	Managed Kubernetes provisioning, live DNS / HTTPS certificates, full observability stack, enterprise secret vault, autoscaling cluster operations

5. User Roles

Role	Responsibilities
Patient	Browse doctors and specialties, book appointments, access patient dashboard
Doctor	Manage profile, availability, appointments, and doctor dashboard
Admin	Manage users, specialties, reports, settings, pages, and approvals
DevOps Maintainer	Maintain containers, pipelines, backup procedures, deployment files, and operational documentation

6. Technology Stack

Layer	Technologies
Application	Laravel 12, PHP 8.2, Blade, Tailwind CSS, Alpine.js, Vite
Database	MySQL 5.7 (Docker), SQLite (CI testing)
Containerization	Docker CE, Docker Compose v2, Nginx Alpine
CI/CD	GitLab CI/CD (5 stages: validate → build → test → package → deploy)
Live Deployment	Proxmox VE 8.4.14, Ubuntu 22.04 VM, iptables NAT, public URL
Deployment Prep	Kubernetes YAML manifests (10 files)
Operations	PowerShell backup script, monitoring plan documentation
IaC	Terraform (main.tf, variables.tf, outputs.tf, provider.tf)

7. Non-Functional Requirements

- Reproducible environment setup via Docker Compose
- Automated validation before every release via GitLab CI
- Live application accessible at <http://51.158.200.127:8080>
- Recoverability through database and storage backups
- Clear repository organization and delivery documentation

8. Live Deployment

The application was deployed live on May 26, 2026 on a Proxmox Virtual Environment server:

Property	Value
Public URL	http://51.158.200.127:8080
Proxmox Host	april26-bootcamp-dataops (PVE 8.4.14)
VM	100 / Doctors-Clinic-DevOps-System / Ubuntu 22.04
Stack	Docker Compose: nginx + app (PHP-FPM) + mysql